

Web Checkout Take-Home Assignment

Build a small **self-service checkout** web app for a snack bar. Picture the setting: a customer walks up to a tablet mounted at the counter, browses the menu, builds an order, pays, and walks away. No cashier, just them and the screen. A client app talking to a web API.

That's the whole spec. We've kept it deliberately short. The interesting decisions are yours to make, and how you make them is most of what we're evaluating.

The essence

- The **client** lets someone see the menu (fetched from the API), build up an order, enter payment, and submit it.
- The **API** serves the menu and accepts the order. Payment can be any shape you like and does **not** need to be real or processed. Submitting an order just needs to persist it (file or DB, your call) and return a sensible response.

Everything beyond that is up to you: the shape of the data, the endpoints, the UX, and what happens when things go wrong.

This is self-service

There is no cashier. The person using this is standing at the screen **alone**, probably in a hurry, with no one to ask for help. They will do things you didn't plan for. Keep that person in mind, and the screen they're standing in front of. They are the one you're actually building for.

On using AI

Use it. All of it. Cursor, Claude Code, Copilot, whatever you reach for on a normal day. We are not testing whether you can type an Express route from memory. We build with these tools every day and we expect you to as well.

One rule: **you are responsible for everything you submit.** "The AI wrote it" is not an explanation for code you can't defend. If it's in your repo, it's yours.

Show us the machine, not just the output

We are at least as interested in **how** you built this as in **what** you built.

Anyone can prompt an AI into a working app. The engineers who impress us have a way of building, a machine behind the machine, and it shows. Make yours visible. Give us a window into how you got from an empty folder to a finished thing: what you decided up front, how you drove the tools, what you kept and what you threw away.

Surprise us here. This is the part that separates people.

Stack

- **API:** we work mostly in Python and Node, a preference, not a requirement.
- **Client:** we lean toward React or Vue, but build with what you're fast in.
- Docker, CSS frameworks, and libraries are all fair game.

Submitting

- Push to a public Git repo and send us the link.
- Include whatever a teammate needs to clone it and have it running on a Unix machine without asking you questions.
- Take a few days. Budget the time you'd give a small piece of real work. We're not looking for a lost weekend.

How we'll evaluate it

We won't hand you a checklist. Figuring out what "good" means here is part of the exercise. We care about **judgment**: what you chose to build, what you chose to skip, how it behaves when reality doesn't cooperate, and whether the person at the screen has a good time.

After you submit, we'll sit down together and talk through what you built and why, and then push past it into a problem or two you didn't see coming.